

Estruturas de Dados

Árvores **Rubro-** Negras

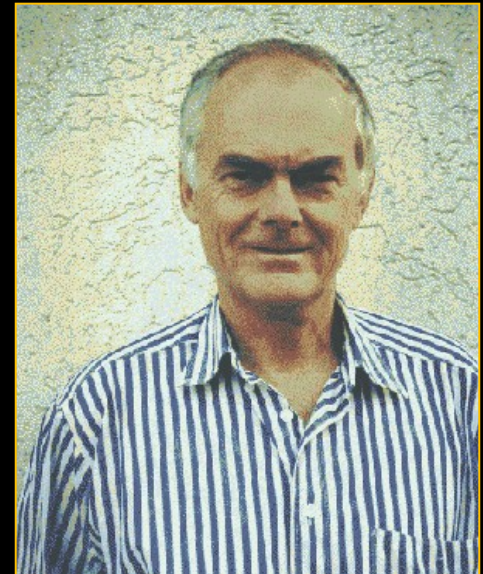
- Características
- Algoritmos

O que são Árvores **Rubro**-Negras

- Árvores de pesquisa binária com um bit de informação adicional: a sua **cor**.
 - Pode ser **rubra** ou **negra**
- Limitação da coração dos nodos da raiz às folhas
 - Garante que nenhum caminho é mais que duas vezes mais longo que qualquer outro
 - Árvore semibalanceada
 - Propriedades comuns a Árvores-B de ordem 4

Histórico

- **Árvores Red-Black** foram inventadas em 1972 por Rudolf Bayer:
 - Estudou matemática em Munique e fez doutorado também em Matemática na Universidade de Illinois, em 1966.
 - Foi pesquisador nos *Boeing Research Labs*, onde inventou em 1970 a **árvore-B**



Publicação Original:

Symmetric binary B-trees: Data structures and maintenance algorithms, *Acta Informatica*, 1 (1972) 290-306

Propriedades Rubro-Negras

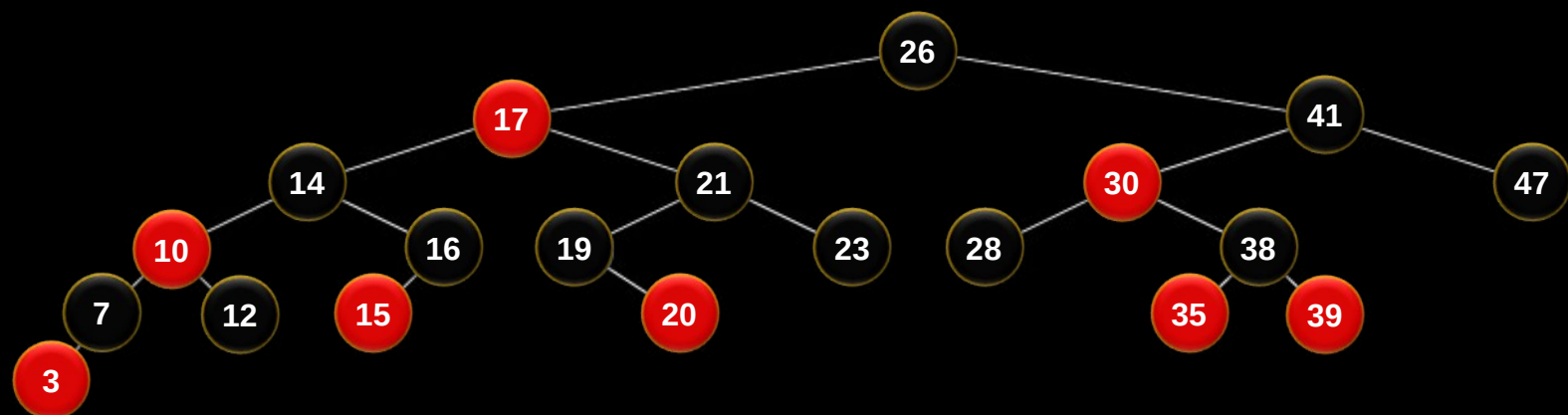
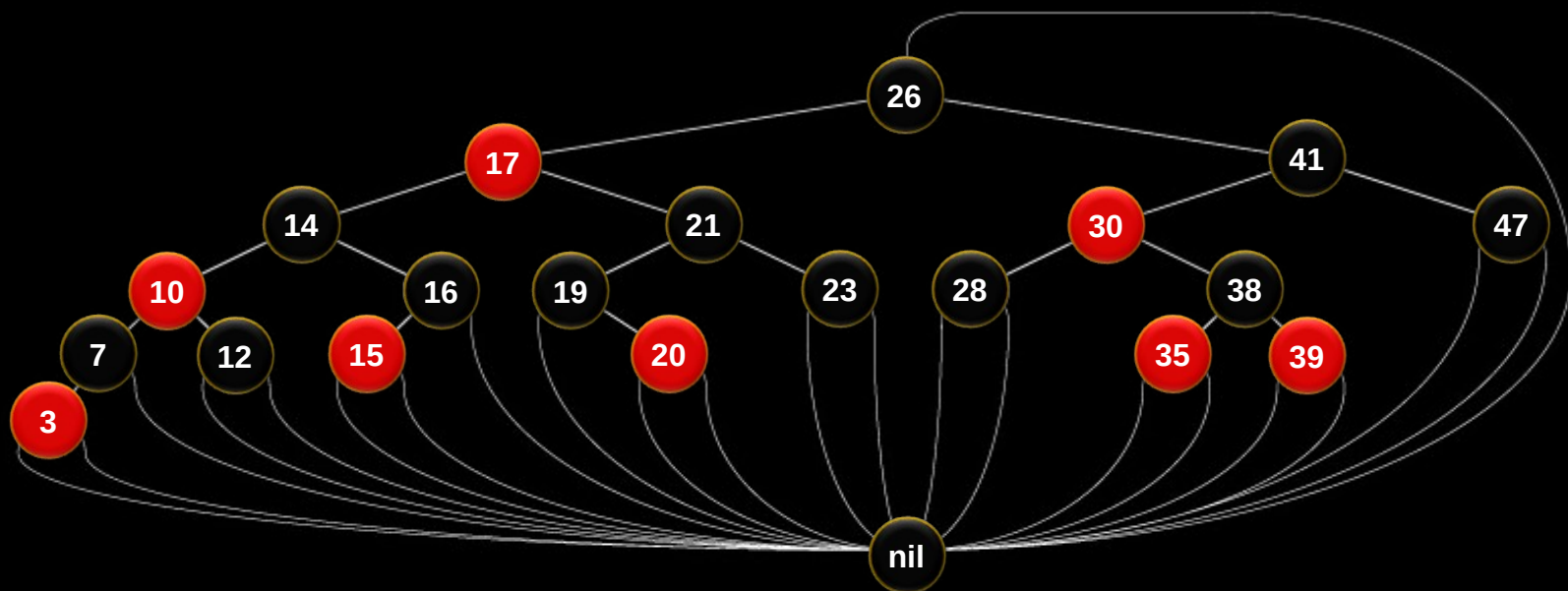
1. Todo nodo ou é **rubro** ou é negro
2. A raiz é negra
3. Toda folha é negra
 - Folhas são somente os nodos vazios (ponteiros nulos).
4. Se um nodo for **rubro**, então ambos os filhos são negros.
5. Para todo nodo, todos os caminhos até uma folha contém o **mesmo número** de nodos negros.

Implementação **Rubro**-Negra

- Nodo possui dois campos a mais: pai e cor

```
Classe nodoRB {  
    info      :      tipo_info;  
    esq       :      *nodoRB;  
    dir       :      *nodoRB;  
    pai       :      *nodoRB;  
    cor       :      {rubro, negro};  
};
```

- Folhas nulas obrigatórias (“toda folha é negra”)
 - Pode-se empregar um único **nodo sentinela** para representar todas as folhas
 - Para fins de visualização em nossos algoritmos vamos ignorar as folhas.

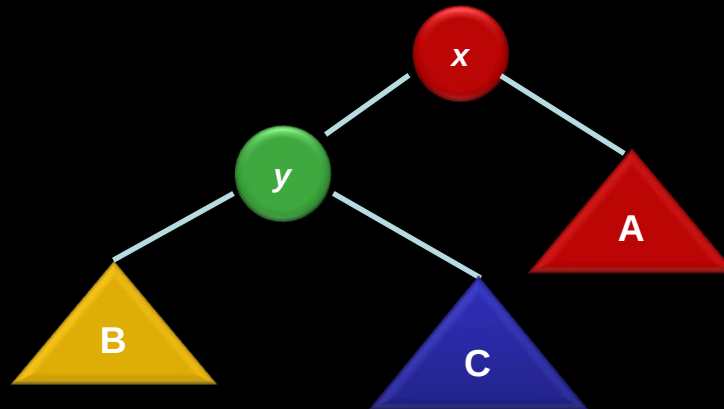


Rotações **Rubro**-Negras

- Árvores Rubro-Negras também realizam rotações para efetuar ajustes
- O Procedimento é quase idêntico à rotação simples das árvores AVL

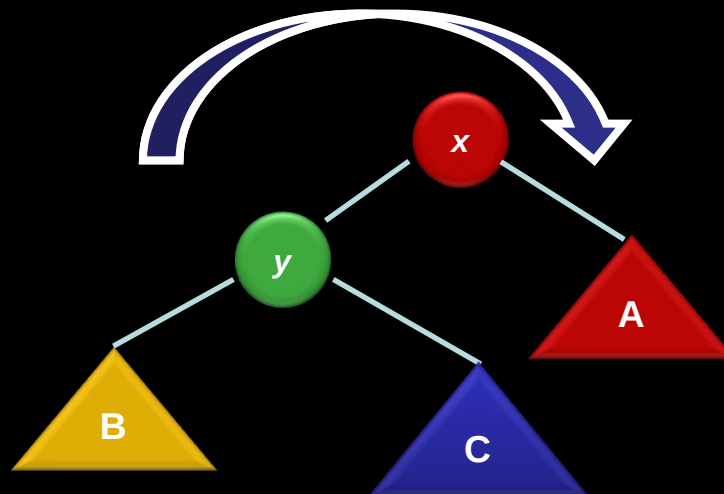
Rotação R^B para a direita

- Corresponde a uma rotação entre os nodos x e seu filho da esquerda, y , envolvendo as subárvores A , B e C



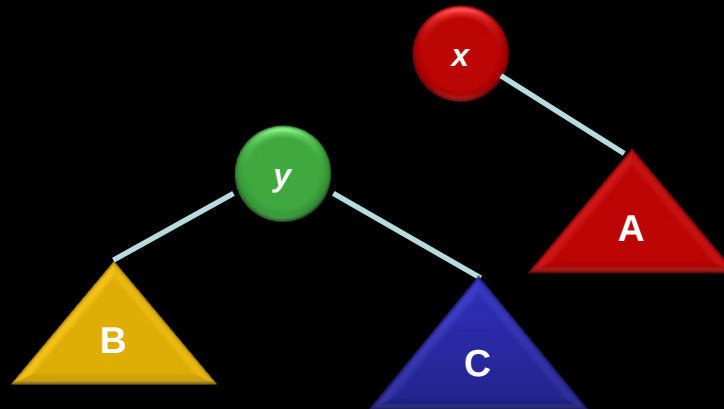
Rotação R^B para a direita

- Corresponde a uma rotação entre os nodos x e seu filho da esquerda, y , envolvendo as subárvores A , B e C



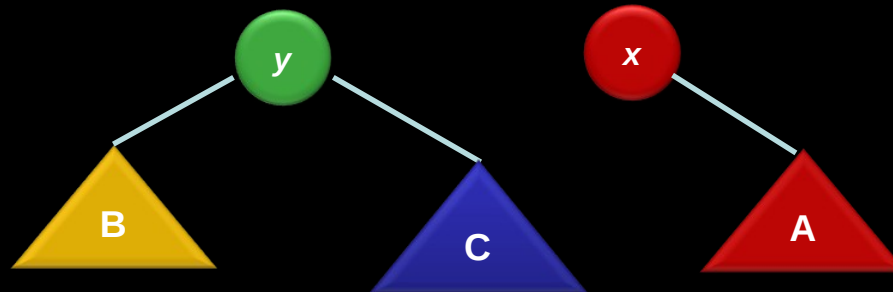
Rotação R^B para a direita

- Corresponde a uma rotação entre os nodos x e seu filho da esquerda, y , envolvendo as subárvores A , B e C



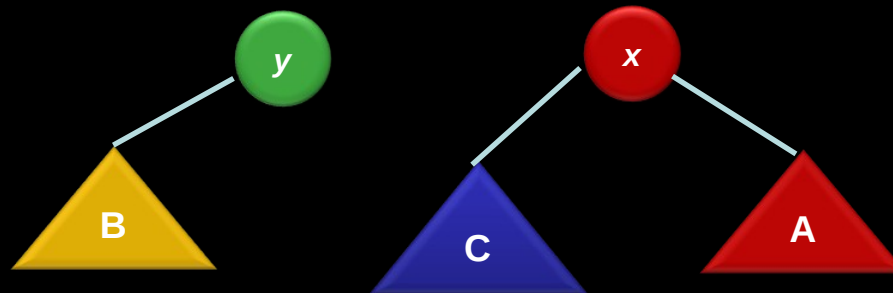
Rotação R^B para a direita

- Corresponde a uma rotação entre os nodos x e seu filho da esquerda, y , envolvendo as subárvores A , B e C



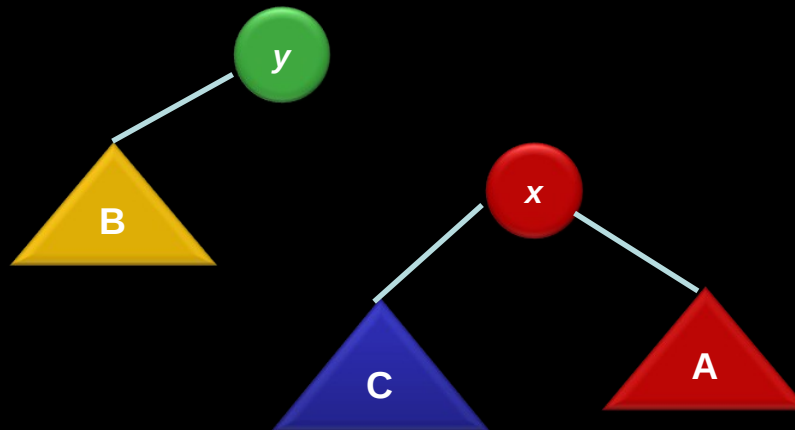
Rotação R^B para a direita

- Corresponde a uma rotação entre os nodos x e seu filho da esquerda, y , envolvendo as subárvores A , B e C



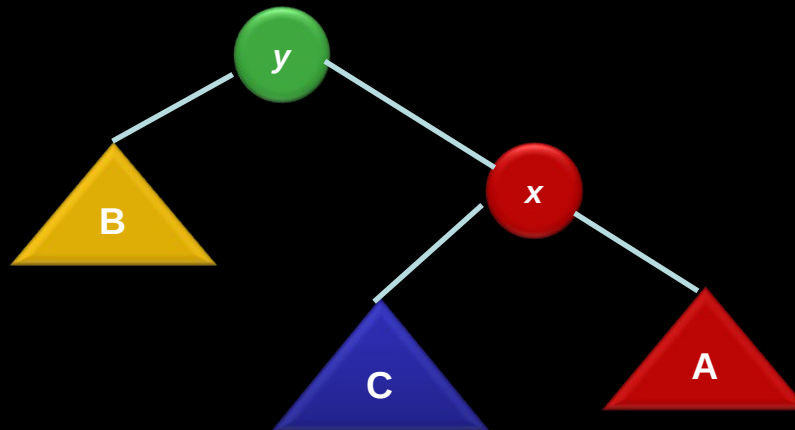
Rotação R^B para a direita

- Corresponde a uma rotação entre os nodos x e seu filho da esquerda, y , envolvendo as subárvores A , B e C



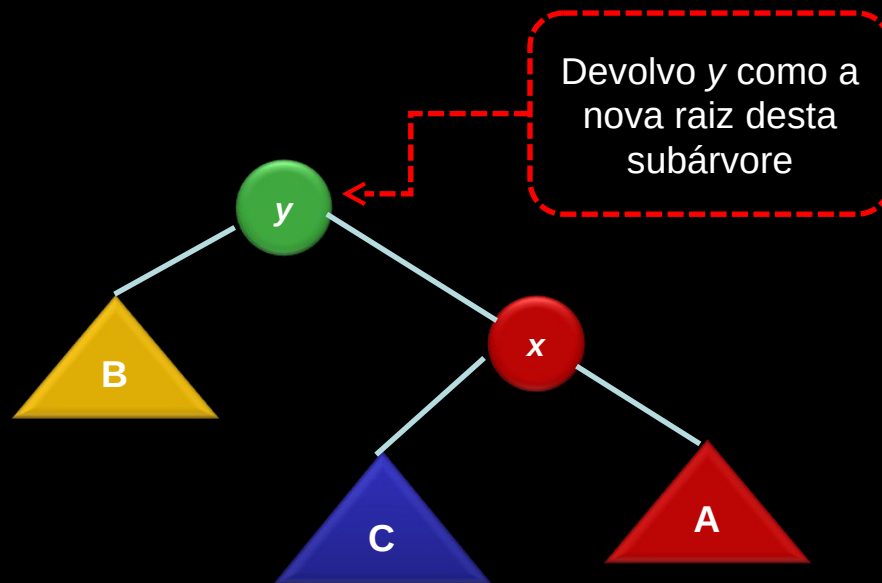
Rotação R^B para a direita

- Corresponde a uma rotação entre os nodos x e seu filho da esquerda, y , envolvendo as subárvores A , B e C



Rotação R^B para a direita

- Corresponde a uma rotação entre os nodos x e seu filho da esquerda, y , envolvendo as subárvores A , B e C



Rotação **RB** para a direita

```
nodorB *roda_dir(x *nodorB)
```

```
  variáveis locais
```

```
    y, pai : *nodorB;
```

```
    lado   : {esq, dir};
```

```
início
```

```
  pai <- x->pai;      //guardo o pai da subárvore que vou rodar
```

```
  SE pai->esq = x ENTÃO //x está pendurado de qual lado do pai?
```

```
    lado <- esq;
```

```
  SENÃO
```

```
    lado <- dir;
```

```
  FIMSE
```

```
  y <- x->esq;
```

```
  y->pai <- pai
```

```
  //roda & ajusta pais
```

```
  x->esq <- y->dir;
```

```
  x->esq->pai <- x;
```

```
  y->dir <- x;
```

```
  y->dir->pai <- y;
```

```
  SE lado = esq ENTÃO
```

```
    pai->esq <- y;
```

```
  SENÃO
```

```
    pai->dir <- y;
```

```
  FIMSE
```

```
  retorne y;
```

```
fim roda_dir
```

Rotação **RB** para a esquerda

```
nodorB *roda_esq(x *nodorB)
```

```
  variáveis locais
```

```
    y, pai : *nodorB;
```

```
    lado    : {esq, dir};
```

```
início
```

```
  pai <- x->pai;    //guardo o pai da subárvore que vou rodar
```

```
  SE pai->esq = x ENTÃO //x está pendurado de qual lado do pai?
```

```
    lado <- esq;
```

```
  SENÃO
```

```
    lado <- dir;
```

```
  FIMSE
```

```
  y <- x->dir;    y->pai <- pai    //roda & ajusta pais
```

```
  x->dir <- y->esq;    x->dir->pai <- x;
```

```
  y->esq <- x;    y->esq->pai <- y;
```

```
  SE lado = esq ENTÃO
```

```
    pai->esq <- y;
```

```
  SENÃO
```

```
    pai->dir <- y;
```

```
  FIMSE
```

```
  retorne y;
```

```
fim roda_esq
```

Inserções **Rubro**-Negras

- A inclusão é sempre executada utilizando-se o algoritmo de inserção em árvore de busca
 - Modificado para manter os ponteiros para o pai.
- Todo novo nodo é colorido **rubro**.
 - Sempre ganha duas folhas nulas negras
- Se o pai do novo nodo for negro
 - Está pronto, trata-se de uma árvore-**RB**
- Se o pai for **rubro**
 - A propriedade 4 está ferida
 - Até duas rotações devem ser realizadas

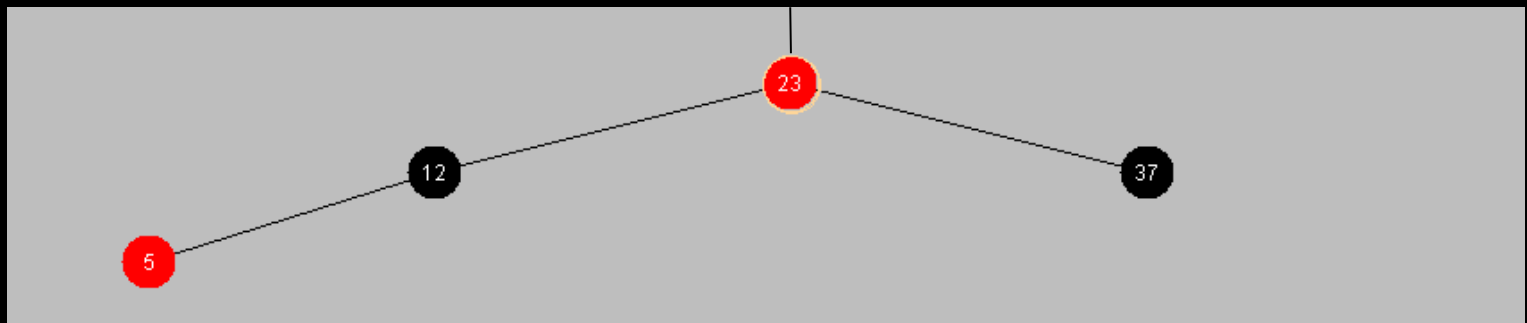
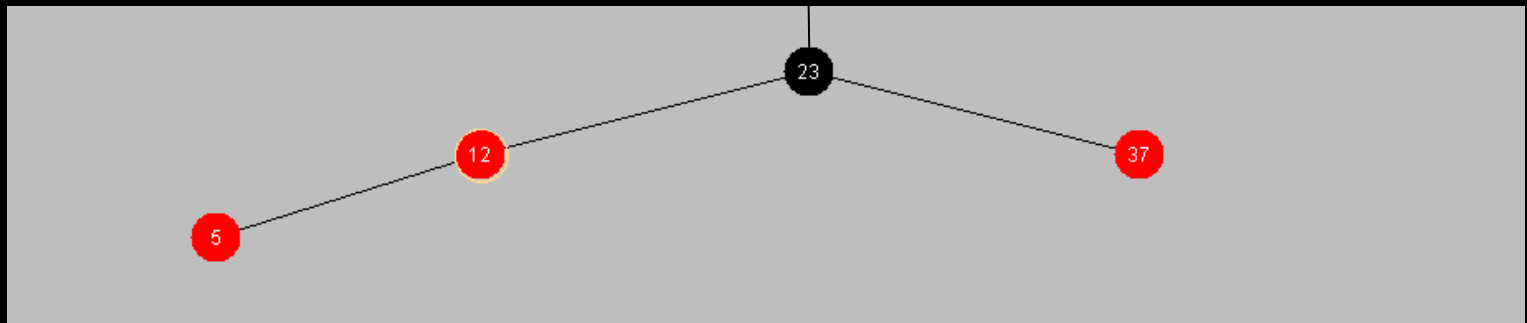
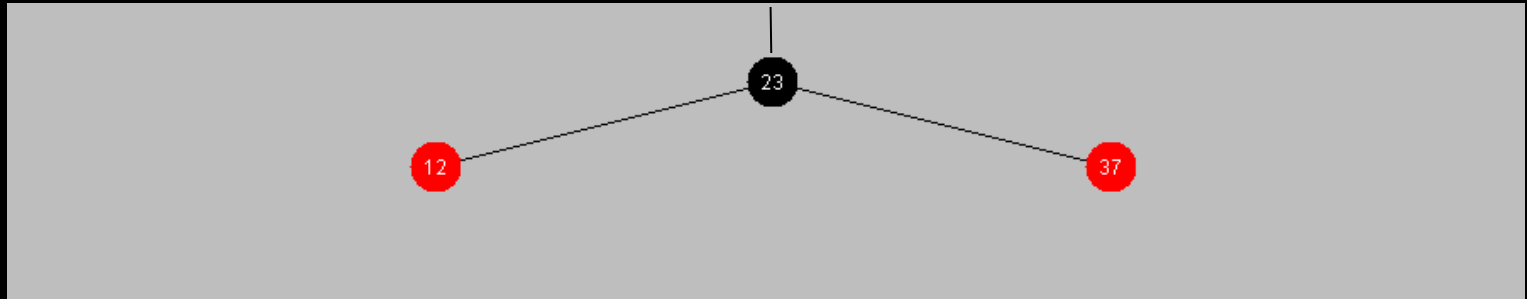
Ajustes Rubro-Negros

- Para todo nodo *v* definimos uma *vizinhança-RB*
 - *v*, seu *pai*, seu *avô* e seu *tio* (RB é binária, haverá no máximo 1 tio)
- Um *passo-CE* de *correção-ou-elevação* envolve
 - *recolorir* e
 - (eventualmente) *rotacionar* a vizinhança-RB
- Um *passo-CE* provoca
 - a *restituição* da propriedade 4 ou
 - a *elevação* do erro para um nível acima na árvore-RB
- Para todo *passo-CE*
 - existem 6 possíveis casos
 - 3 para a direita e 3 para a esquerda

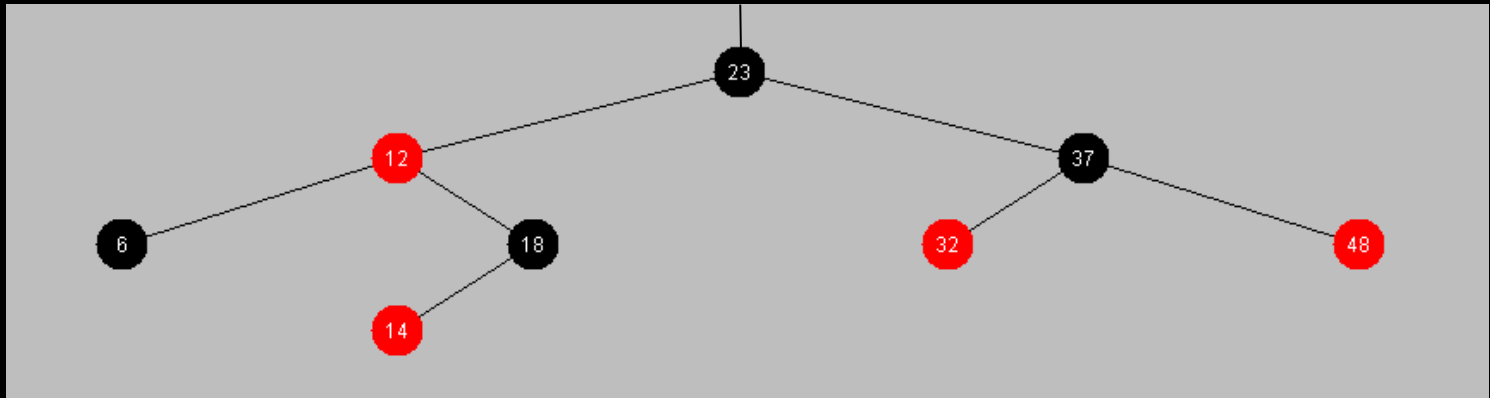
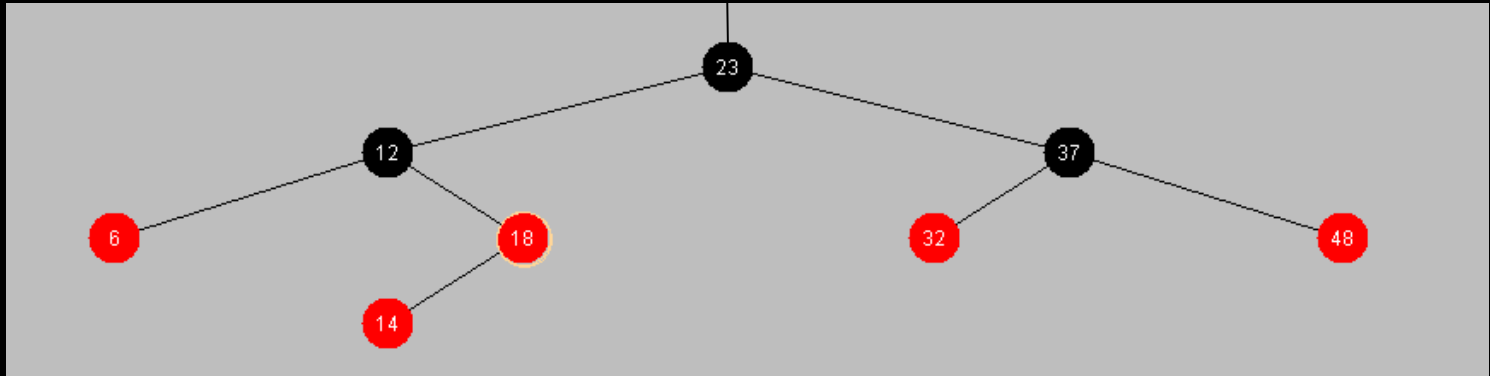
Caso 1e (esquerda)

- v possui um tio **rubro** e seu pai é um filho **rubro** da esquerda
 - não importa se v é filho da esquerda ou direita
- recolorimos o pai, avô e tio de v
 - se o bisavô de v for negro está feito
- caso contrário a violação de 4 foi elevada 2 níveis
 - v é elevado dois níveis
 - executamos um *passo-CE* para o novo v (avô de v)

Caso 1e (esquerda)



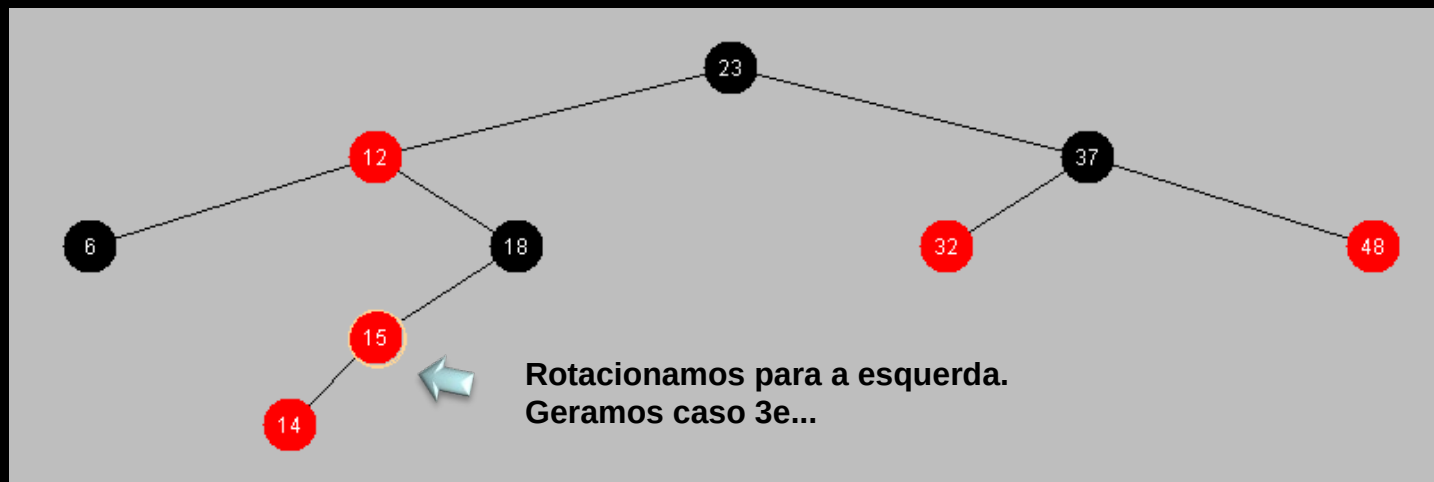
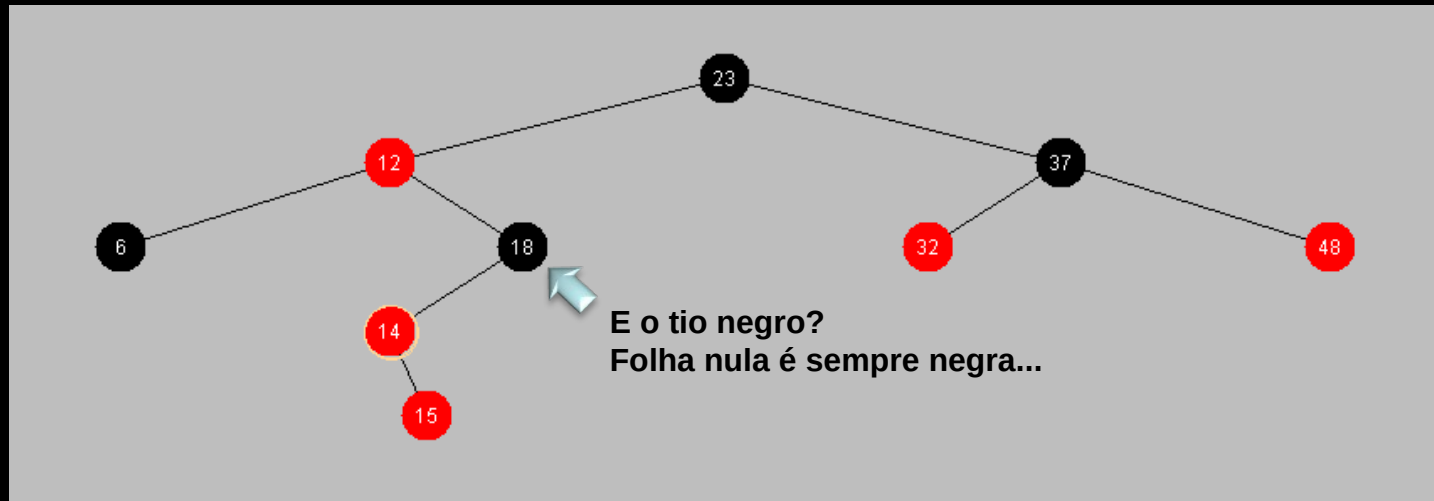
Caso 1e (inserção do 14 - esquerda)



Caso 2de (direita-esquerda)

- v é um filho da direita, cujo pai é um filho **rubro** da esquerda e cujo tio é negro
- rotacionamos para a esquerda v e seu pai
 - atualizamos o nodo corrente
 - caímos no caso 3

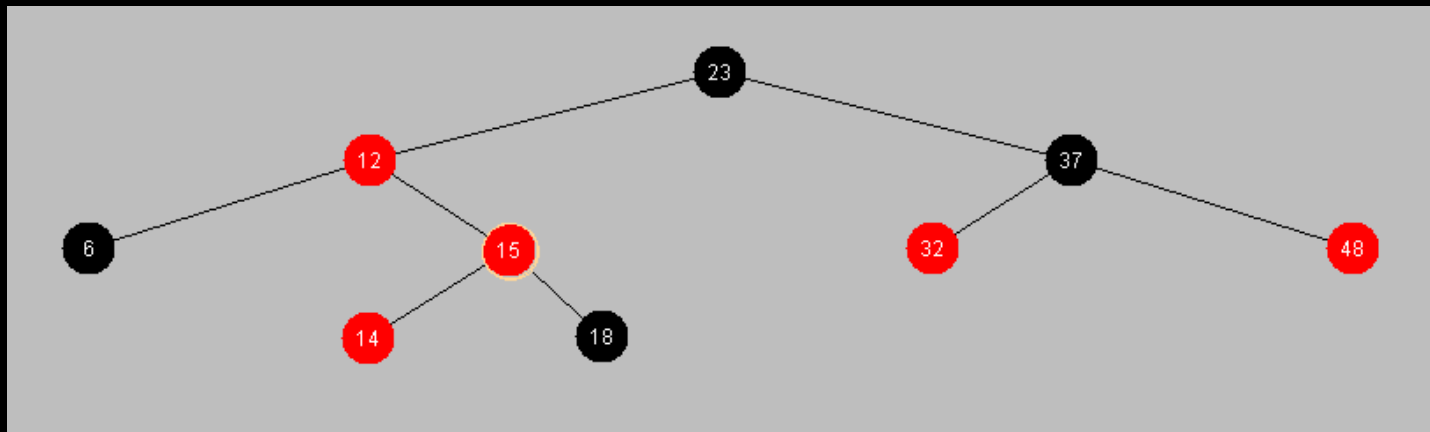
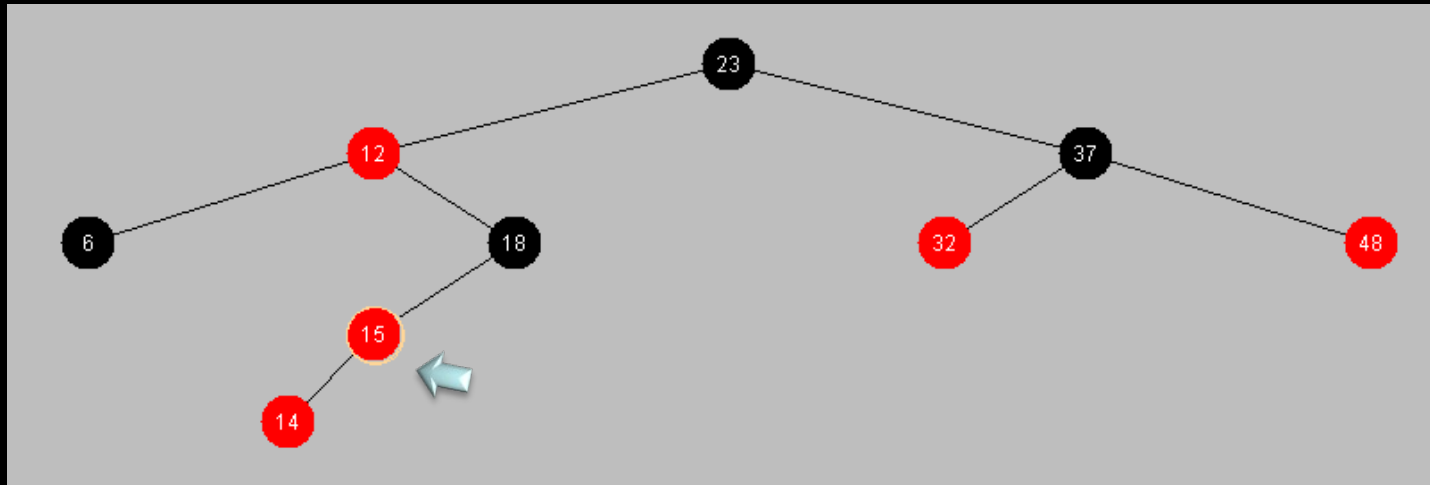
Caso 2de (inserção do 15 - direita-esquerda)



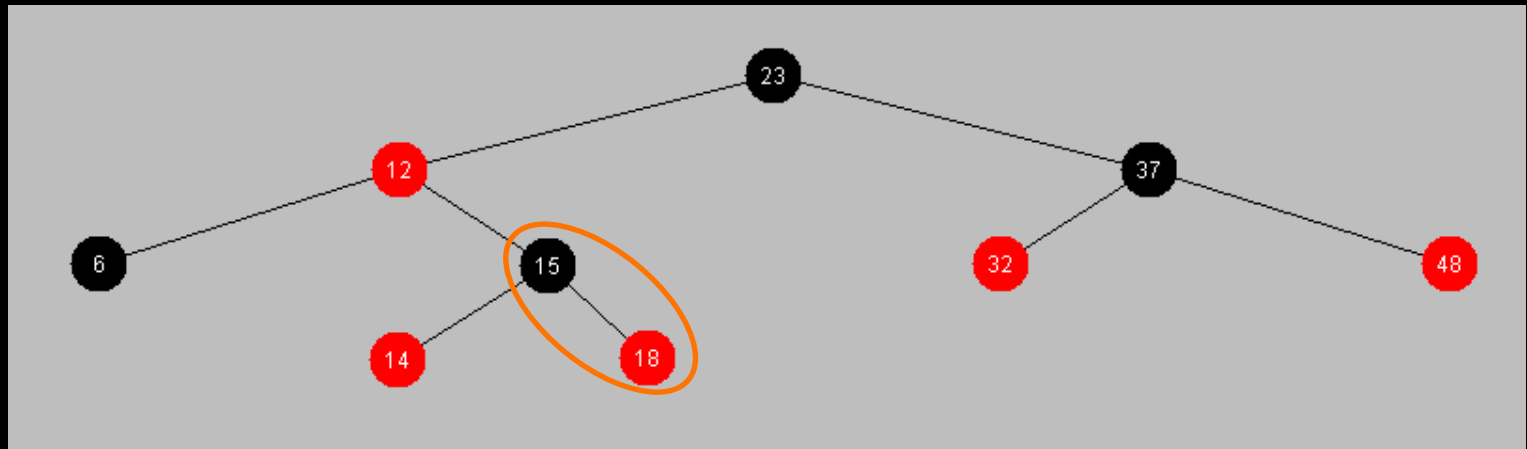
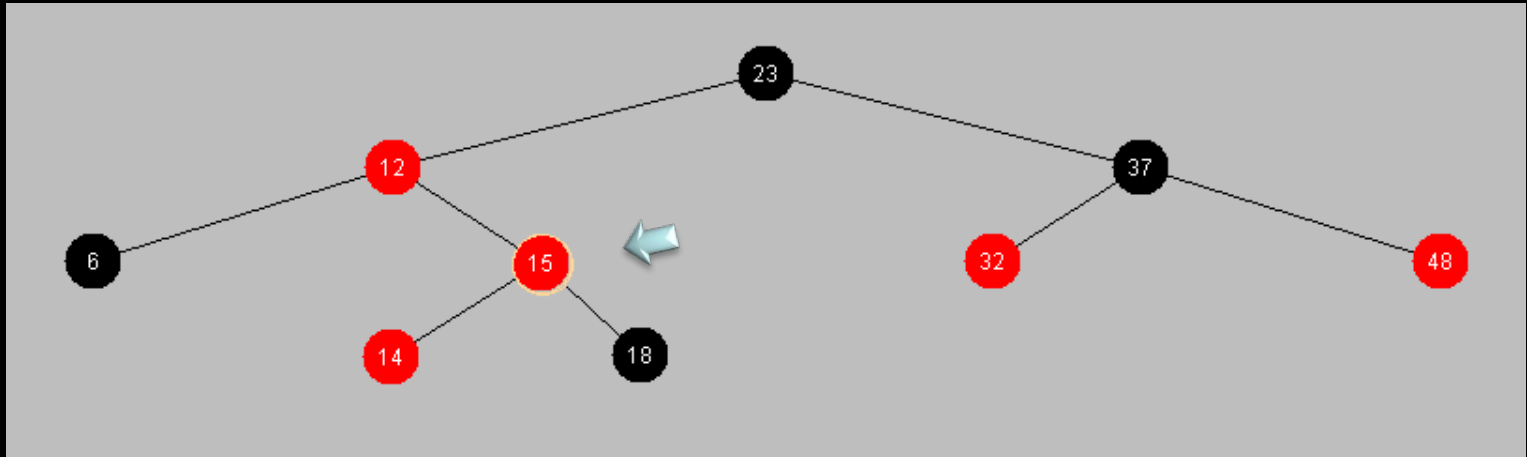
Caso 3e (esquerda)

- v é um filho da esquerda, cujo pai é um filho **rubro** da esquerda e cujo tio é negro
- rotacionamos para a direita entre o avô e o pai
- recolorimos o pai e o avô
 - pai fica negro
 - avô fica **rubro**

Caso 3e

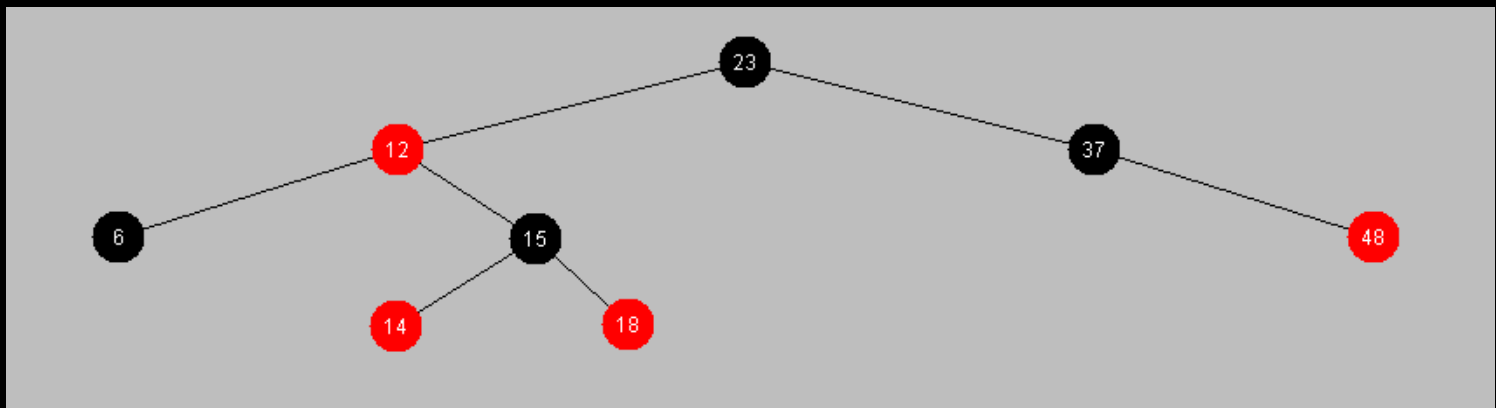


Caso 3e



Deleção do 32

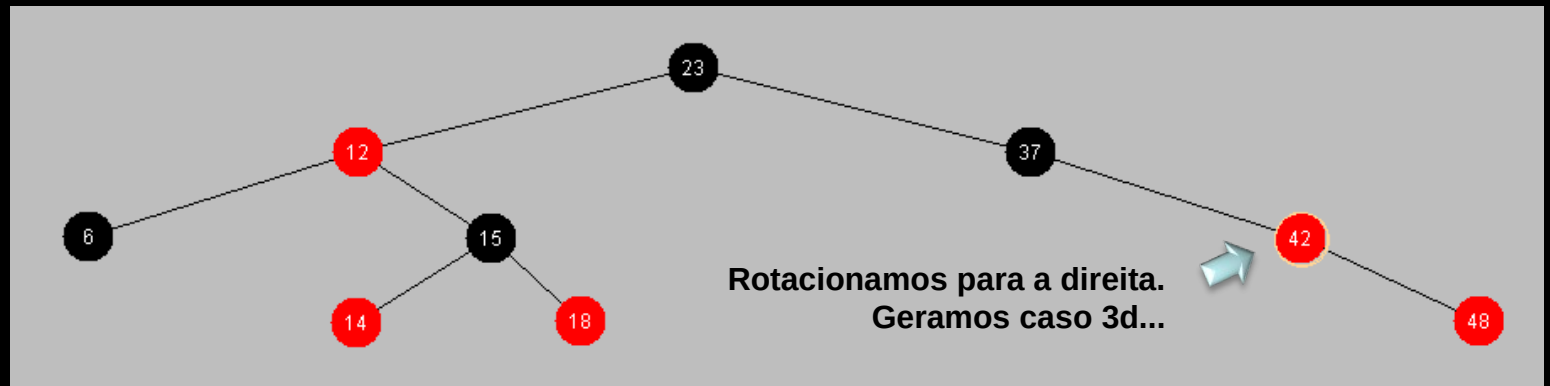
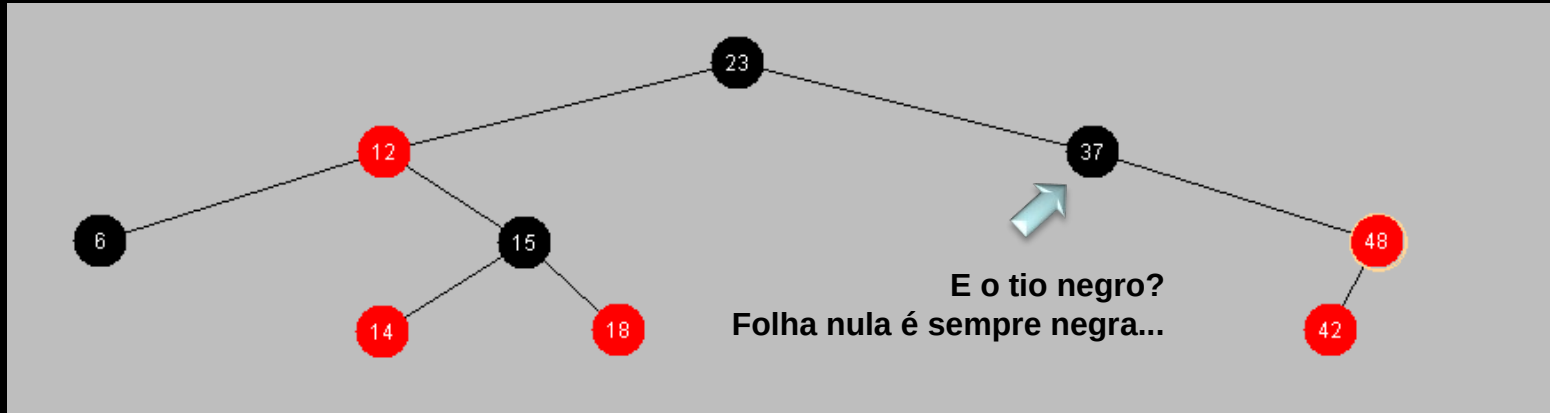
- É deletado como na árvore de busca não balanceada.
 - Caso simples: basta apagar o nodo.
- Fizemos aqui apenas para usar a árvore resultante para ver mais exemplos de inserção.
 - Veremos deleção mais adiante



Caso 2ed (esquerda-direita)

- v é um filho da esquerda, cujo pai é um filho **rubro** da direita e cujo tio é negro
- rotacionamos para a direita v e seu pai
 - atualizamos o nodo corrente
 - caímos no caso 3

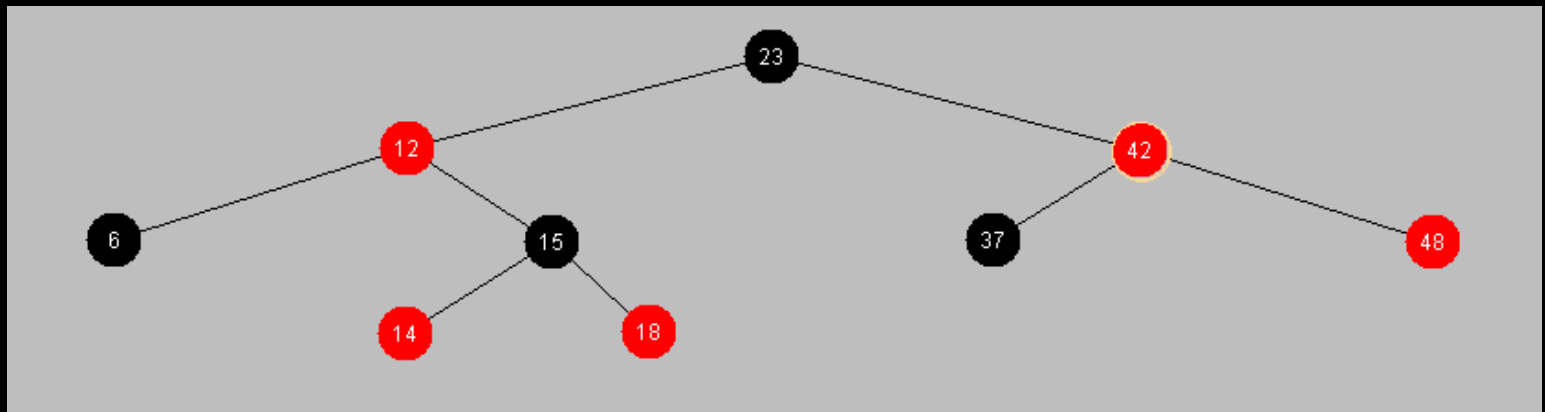
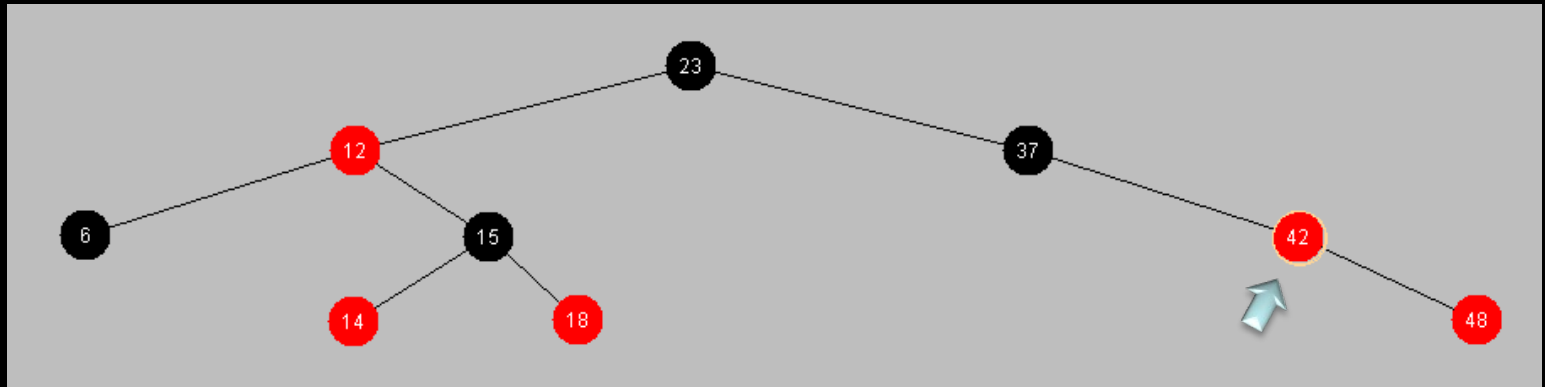
Caso 2ed (inserção do 42 – esquerda-direita)



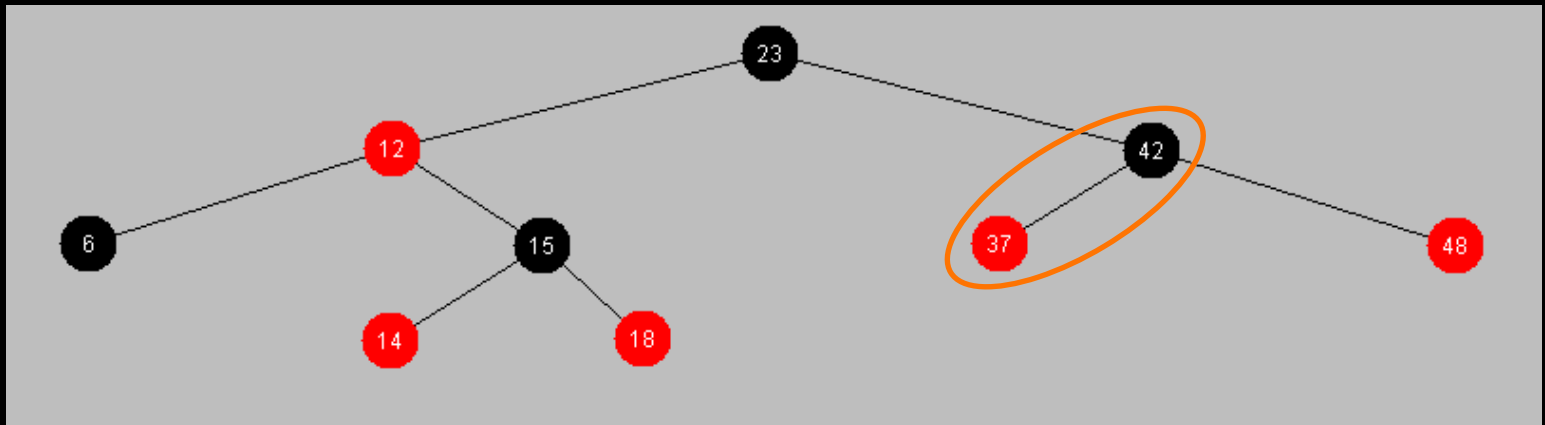
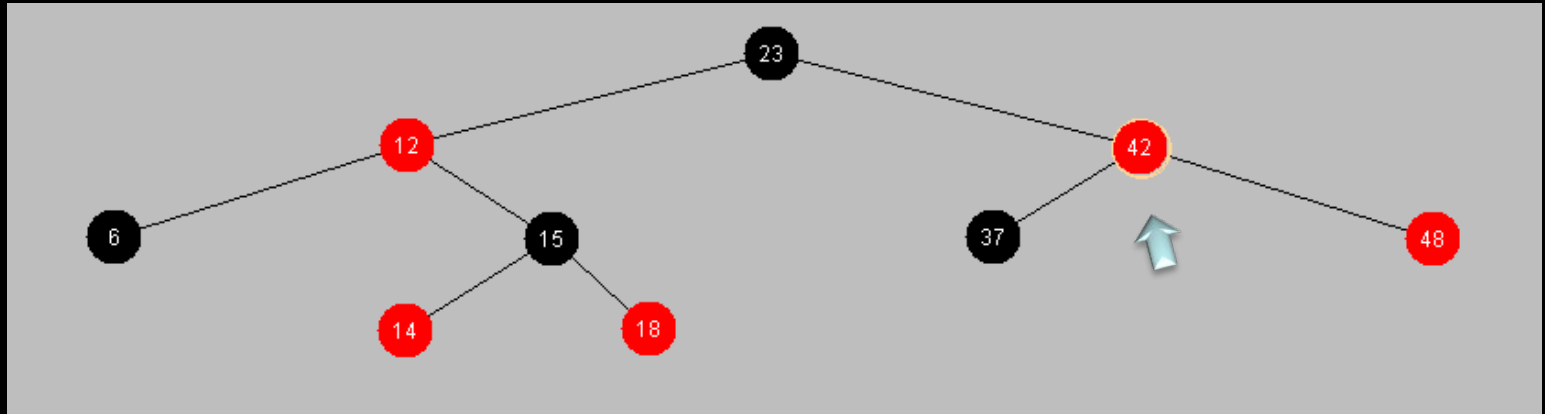
Caso 3d (direita)

- v é um filho da direita, cujo pai é um filho **rubro** da direita e cujo tio é negro
- rotacionamos para a esquerda entre o avô e o pai
- recorimos o pai e o avô
 - pai fica negro
 - avô fica **rubro**

Caso 3d



Caso 3d



Inserção-RB

```
InsererRB (nodoRB *raiz, tipoInfo info, nodoRB *nodo)
    nodoRB    *atual, *pai, *avô;
```

início

```
    InsereArvBusca(raiz, info, nodo); //mod.p/atualizar ref.p/pai
```

```
    nodo->cor <- rubro;
```

```
    atual <- nodo;
```

```
    ENQUANTO (atual ≠ raiz E atual->pai->cor = rubro) FAÇA
```

```
        pai <- atual->pai;
```

```
        avô <- pai->pai;
```

```
        SE avo->esq = pai ENTÃO
```

```
            PassoCE_esq(atual, pai, avô);
```

```
        SENÃO
```

```
            PassoCE_dir(atual, pai, avô);
```

```
        FIMSE
```

```
    FIM ENQUANTO
```

```
    raiz->cor <- negro; //caso zero, por via das dúvidas
```

fim.

Passo-CE-RB

PassoCE_esq (nodoRB *atual, *pai, *avô)

nodoRB *tio;

início

tio <- avô->dir;

SE tio->cor = **rubro** **ENTÃO** //caso 1

 pai->cor <- negro;

 avô->cor <- **rubro**;

 tio->cor <- negro;

 atual <- avô; //sobe 2 níveis

SENÃO

SE pai->dir = atual **ENTÃO** //caso 2

 roda_esq(pai);

 atual <- atual->esq; //reseta atual

FIMSE

 pai->cor <- negro; //caso3

 avô->cor <- **rubro**;

 roda_dir(avô);

FIMSE

fim PassoCE-esq

Passo-CE-RB

PassoCE_dir (nodoRB *atual, *pai, *avô)

nodoRB *tio;

início

tio <- avô->esq;

SE tio->cor = **rubro** **ENTÃO** //caso 1

 pai->cor <- negro;

 avô->cor <- **rubro**;

 tio->cor <- negro;

 atual <- avô; //sobe 2 níveis

SENÃO

SE pai->esq = atual **ENTÃO** //caso 2

 roda_dir(pai);

 atual <- atual->dir; //reseta atual

FIMSE

 pai->cor <- negro; //caso3

 avô->cor <- **rubro**;

 roda_esq(avô);

FIMSE

fim PassoCE-dir

Deleções **Rubro**-Negras

- A deleção do nodo **v** é sempre executada utilizando-se o algoritmo de deleção em árvore de busca
 - Modificado para manter os ponteiros para o pai
 - Sempre tentamos subir **w**, o sucessor inordem de **v**
 - menor elemento da subárvore direita
- Ao subir a chave **w** nunca subimos junto a cor de **w**
 - o nodo cujo valor foi “retirado” mantém a cor original, associada à chave **v**
- Se **w** for **rubro**, a árvore-RB continua válida
- Se **w** for negro e possuir um filho **rubro**
 - **recolorimos**

Deleções **Rubro**-Negras

- Problema ocorre quando **w** e seus filhos são todos negros
 - deleção de **w** fere propriedade 5
 - haverá um caminho com um nodo negro a menos
- Realizaremos recolorações e rotações para restaurar propriedades-RB
 - **vizinhança-RB** consistirá do nodo atual, seu pai, seu irmão e os sobrinhos (filhos do irmão)
 - recoloração e rotação ou restituirá a propriedade 5 ou elevará a violação 1 nível na árvore: **passo-CED**
- Para todo **passo-CED** haverá 8 casos
 - 4 para a esquerda e 4 para a direita

Deleções **Rubro**-Negras (esquerda)

Caso 1 - Irmão **y** de **w** é rubro

- Transforma-se em caso 2, 3 ou 4

Casos 2, 3 e 4: irmão **y** de **w** é negro

- Distinguem-se dependendo dos filhos de **y**

Caso 2 - ambos os filhos de **y** são negros

- Recolorimos e violação é elevada um nível

Caso 3 - **y**->**esq** é **rubro** e **y**->**dir** é negro

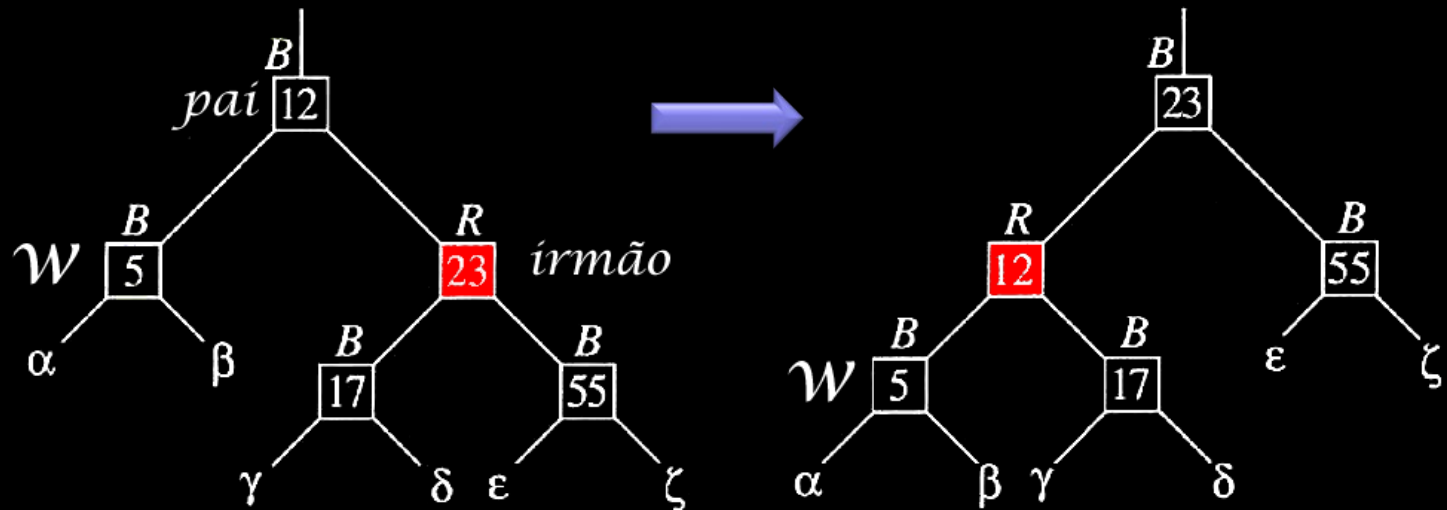
- Recolorimos e rodamos
- É transformado no caso 4

Caso 4 - **y**->**dir** é **rubro**

- Recolorimos e rodamos
- Ajuste termina aqui

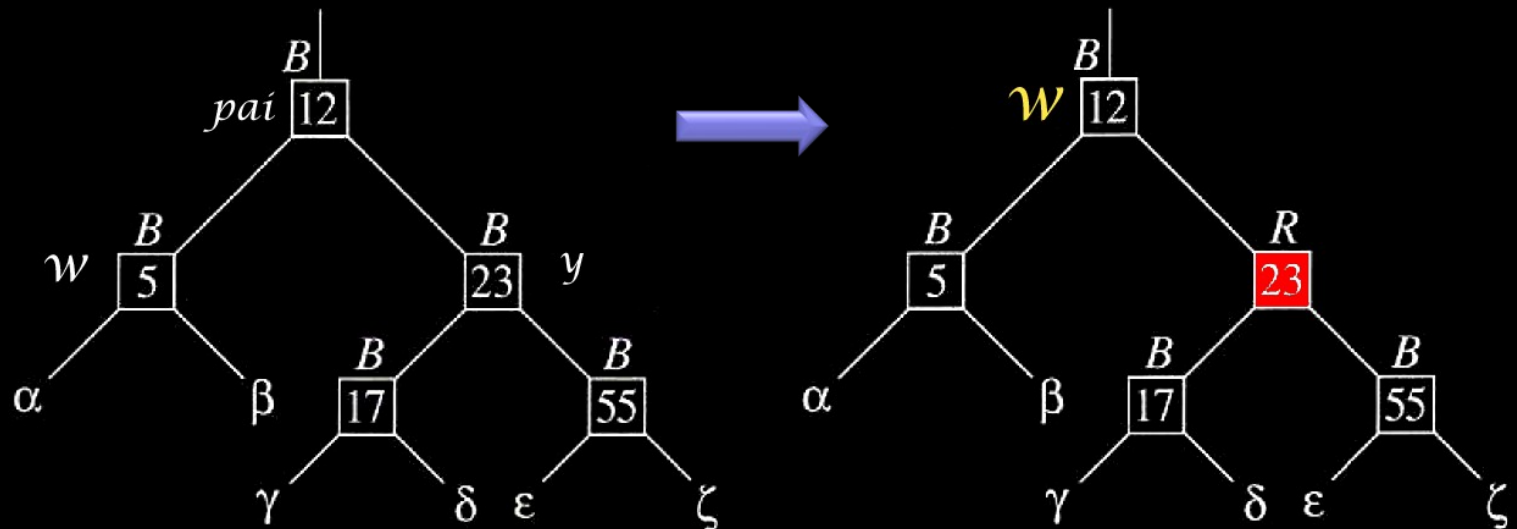
Deleção - Caso 1e (esquerda)

- Se Irmão y de w é rubro
 - troque cor de pai e y
 - rotação para a esquerda entre pai e y



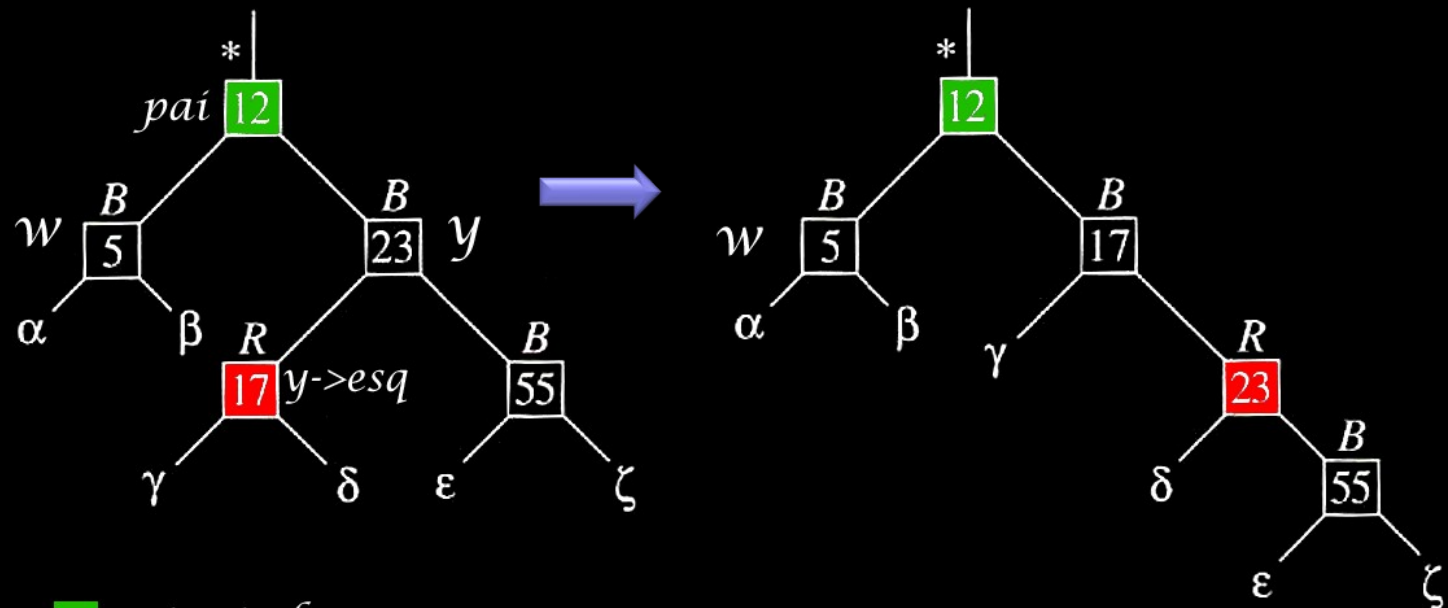
Deleção - Caso 2e (esquerda)

- Se filhos de y são ambos negros
 - troque cor de y para rubro
 - mova w um nível acima



Deleção - Caso 3e (esquerda)

- $y \rightarrow \text{esq}$ é **rubro** e $y \rightarrow \text{dir}$ é negro
 - troque cor de y para **rubro** e de $y \rightarrow \text{esq}$ para negro
 - rotação para direita entre y e $y \rightarrow \text{esq}$

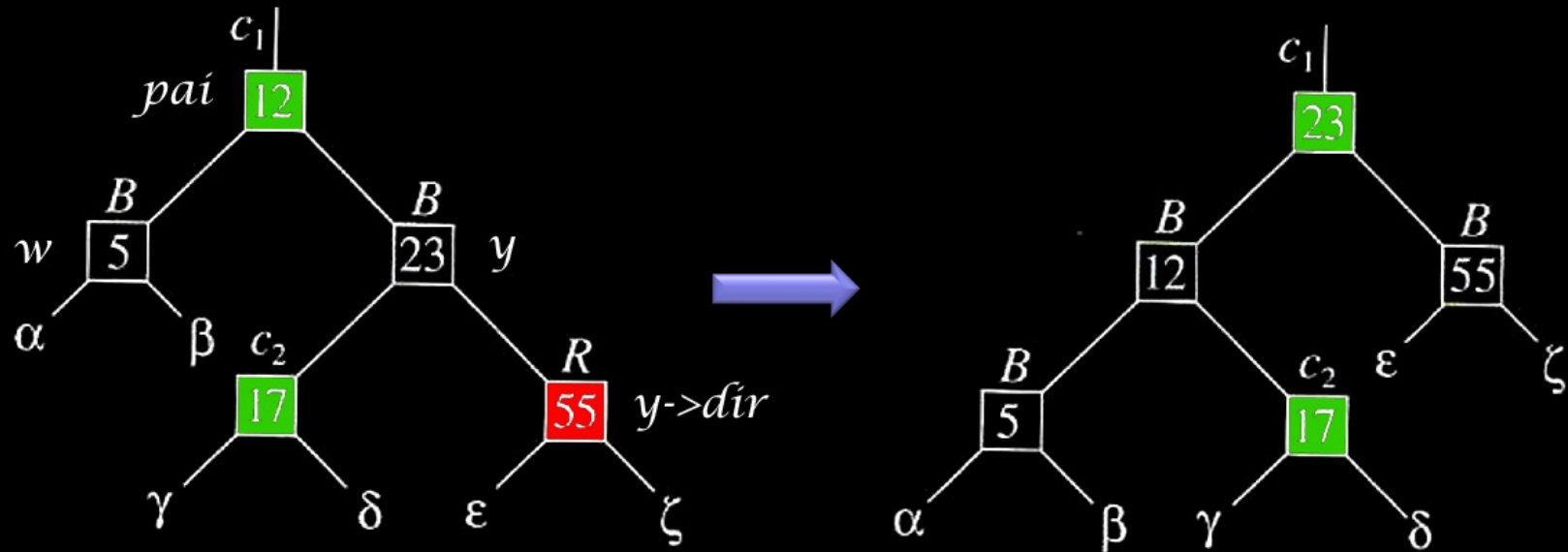


■ -> tanto faz a cor

Deleção - Caso 4e (esquerda)

- $y \rightarrow \text{dir}$ é rubro
 - nodos c_1 e c_2 podem ser rubros ou negros e não mudam
 - mude cor de $y \rightarrow \text{dir}$ e de pai para negro
 - mude cor de y para $c_1 \rightarrow \text{cor}$
 - rotacione para a esquerda entre pai e y
 - faça a raiz da árvore ser w
 - condição de término

Deleção - Caso 4e (esquerda)



Correção-ou-elevação-RB de alto nível

```
nodoRB *passoCED(nodoRB *w, *raiz)
```

```
início
```

```
    ENQUANTO w ≠ raiz FAÇA                                // Cond. de término
```

```
        SE w = w->pai->esq ENTÃO
```

```
            w ← passoCED_esq(w, raiz);                    // Esquerda
```

```
        SENÃO
```

```
            w ← passoCED_dir(w, raiz);                    // Direita
```

```
    FIM ENQUANTO
```

```
    retorne w;
```

```
FIM passoCED
```

Observe que aqui não foi contemplada uma rotina de alto nível que deleta e sobe *w*, onde são tratadas as duas situações onde um passo-CED é desnecessário e que invoca **passoCED()**.

Esta fica por conta do aluno.

```
nodoRB *passoCED_esq(nodoRB *w, *raiz)
variáveis
    nodoRB *y;
início
    y ← w->pai->dir;                                // Irmão da direita
    SE y->cor = RUBRO ENTÃO
        y->cor ← NEGRO;                                // Caso 1
        w->pai->cor ← RUBRO;                            // Caso 1
        roda_esq(w->pai);                              // Caso 1
        y ← w->pai->direita;                            // Reseta Caso 1
    FIMSE
    SE y->esq->cor = NEGRO E y->dir->cor = NEGRO ENTÃO
        y->cor ← RUBRO;                                // Caso 2
        w ← w->pai;                                    // Sobe Caso 2
    SENÃO SE y->dir->cor = NEGRO ENTÃO
        y->esq->cor ← NEGRO;                            // Caso 3
        y->cor ← RUBRO;                                // Caso 3
        roda_dir(y);                                    // Caso 3
        y ← w->pai->dir;                                // Caso 3
    FIMSE
    y->cor ← w->pai->cor;                                // Caso 4
    w->pai->cor ← NEGRO;                                // Caso 4
    y->dir->cor ← NEGRO;                                // Caso 4
    roda_esq(w->pai);                                    // Caso 4
    w ← raiz;                                           // Caso 4
    FIMSE FIMSE
    retorne w;
FIM passoCED_esq
```